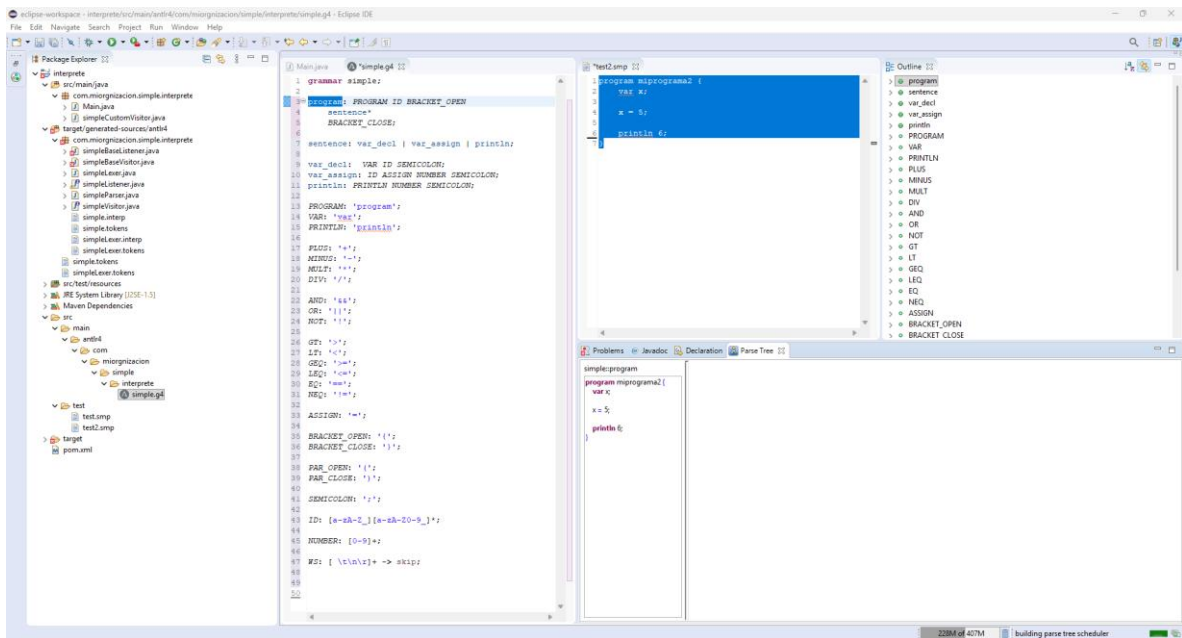
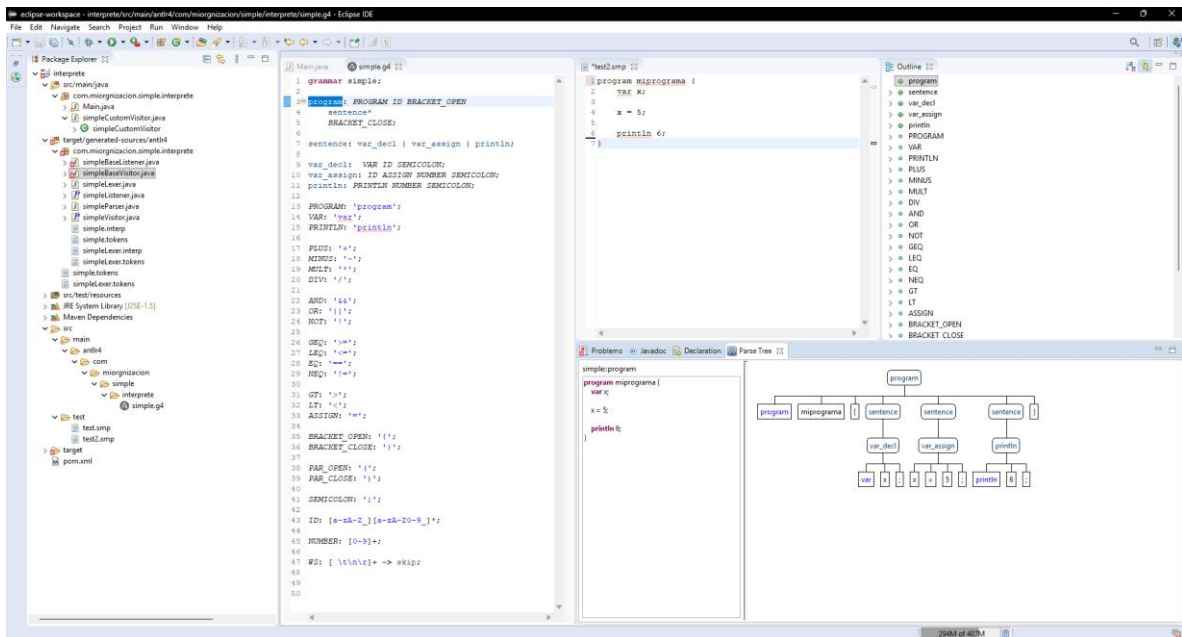
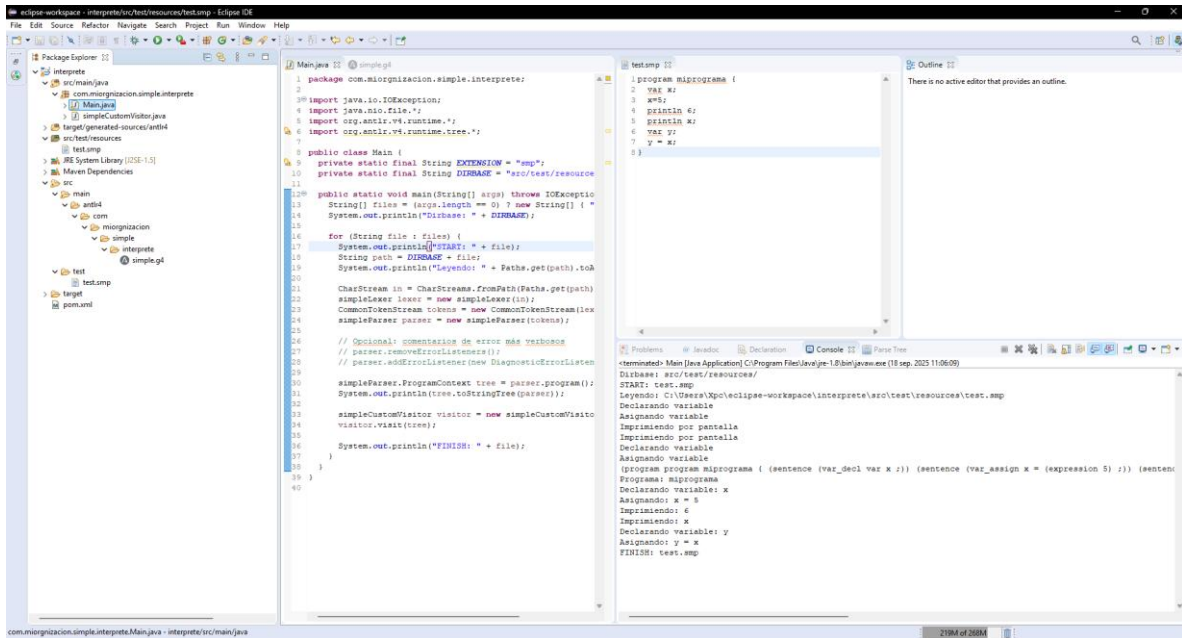




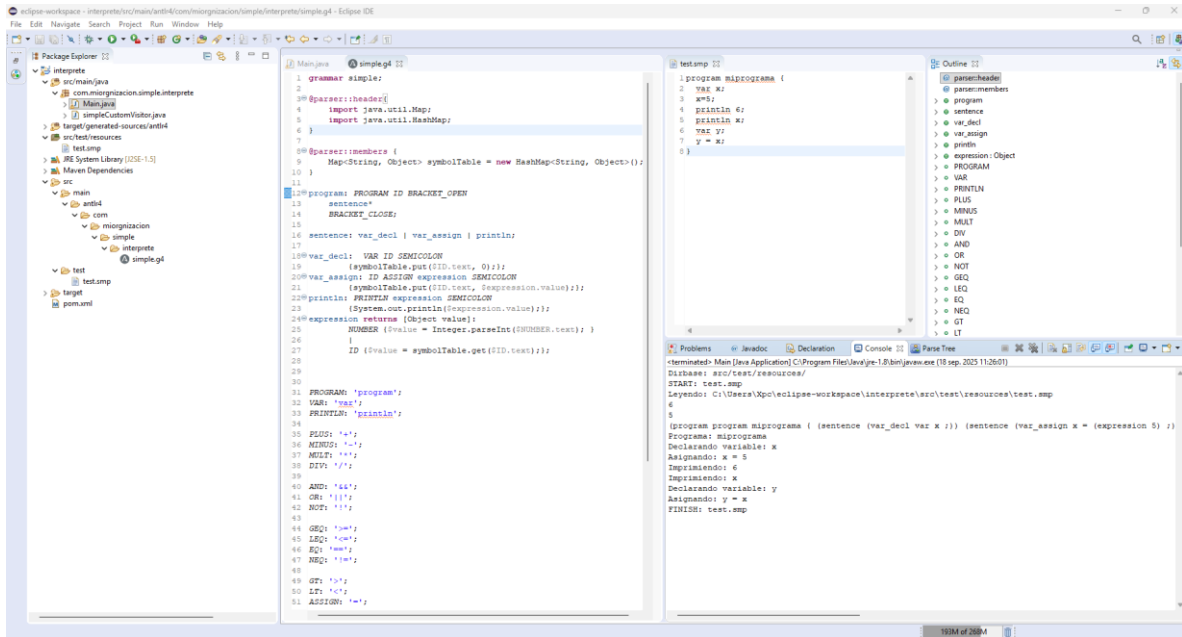
Error a la hora de compilar el parse tree



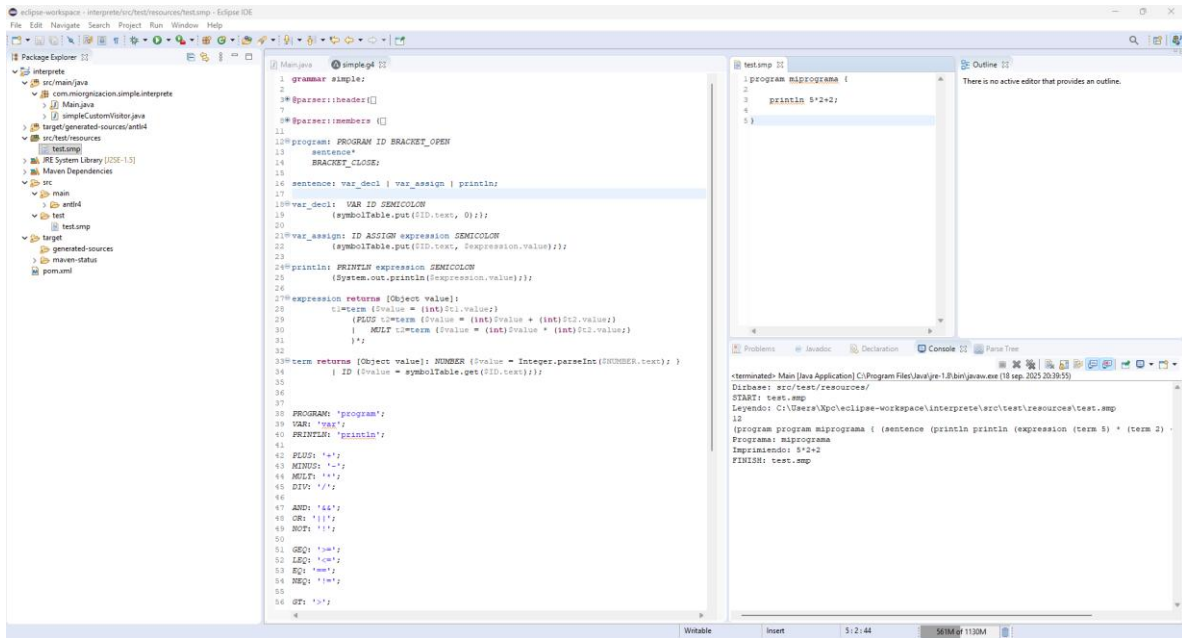
Error solucionado



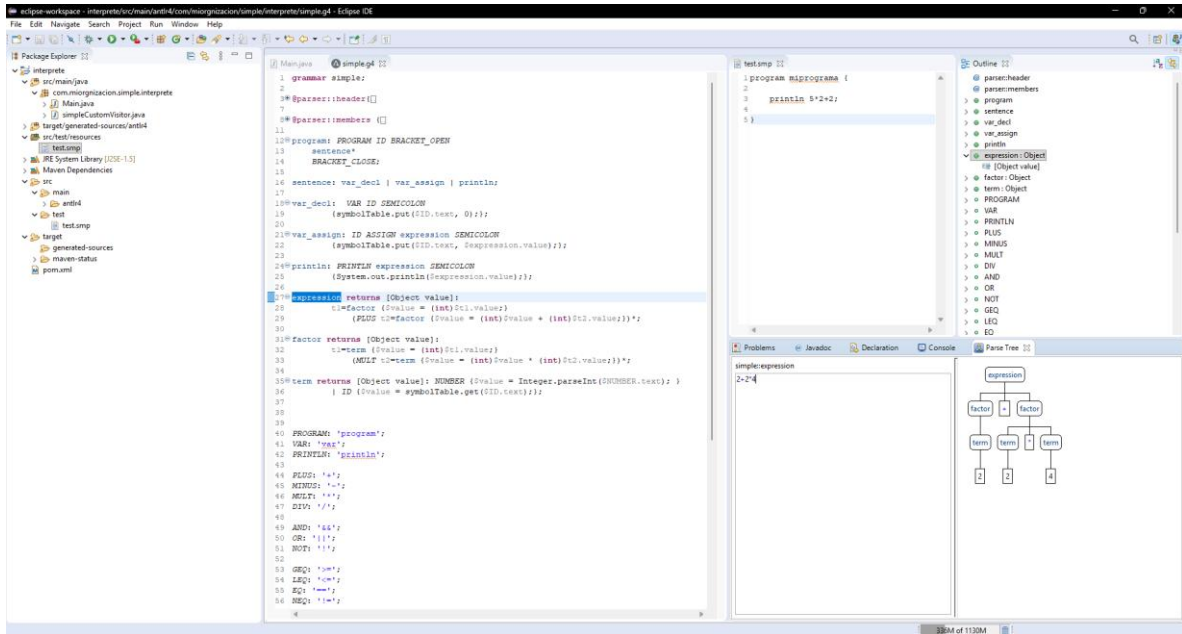
Imprimiendo en pantalla y errores solucionados



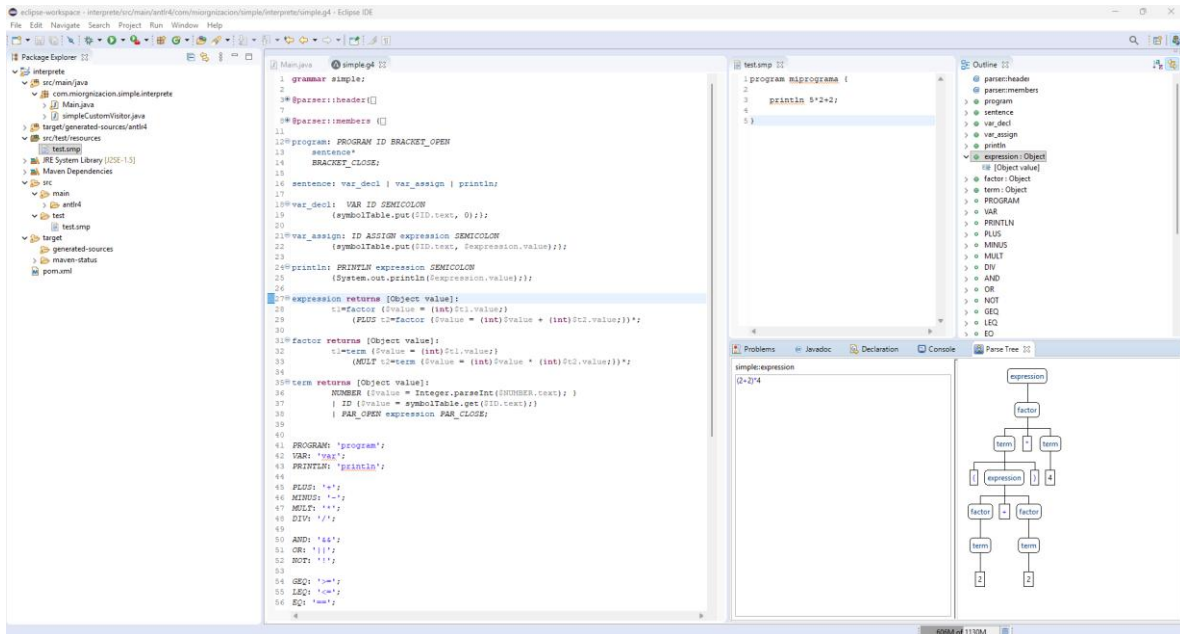
Programa de Analisis semántico funcional.



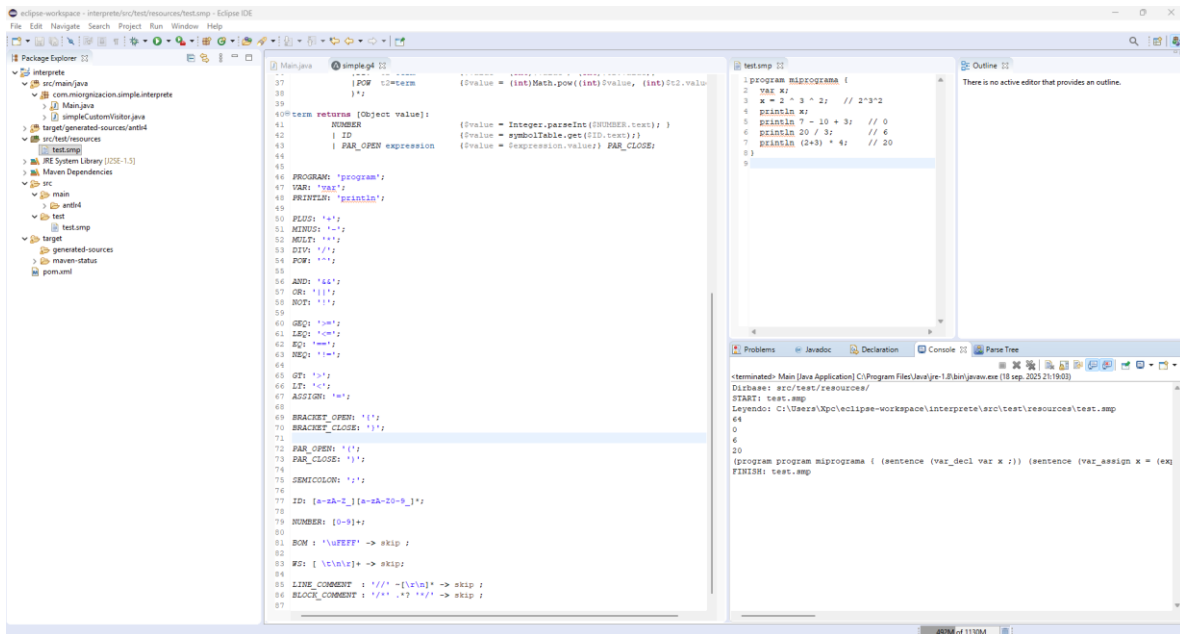
Programa funcional para expresiones aritméticas



Correcto funcionamiento de las expresiones



Modificación para uso de paréntesis



Código completando la tarea del video (operaciones aritméticas faltantes)

Punto 4 — Análisis semántico y Tabla de Símbolos (resumen para informe)

¿Qué es el análisis semántico?

Es la fase del compilador/intérprete que, a partir del árbol sintáctico, verifica el significado del programa: que los identificadores estén declarados, que los tipos sean compatibles en operaciones y asignaciones, y que las reglas de ámbito se cumplan.

En un intérprete, además de verificar, se evalúan expresiones y se producen valores para ejecutar instrucciones (ej. `println`).

Tabla de Símbolos (`symbolTable`)

La `symbolTable` mapea nombres -> símbolos (tipo, valor y metadatos). Para manejar visibilidad se usa una pila de ámbitos:

- Global al iniciar.
- `push` al entrar a un bloque `{ ... }`.
- `pop` al salir del bloque.
- La resolución de nombres busca desde el ámbito más interno hacia afuera.

Operaciones básicas de la `symbolTable`

- `define(nombre, símbolo)`: inserta en el ámbito actual. Error si ya existe ahí.
- `resolve(nombre)`: busca en la pila de ámbitos (de adentro hacia afuera). Error si no existe.
- `assign(nombre, valor)`: resuelve y actualiza el valor del símbolo.

Sistema de tipos (sugerido)

Tipos primitivos: `INT`, `FLOAT`, `BOOL`, `STRING` (opcional `LIST`).

Reglas prácticas:

- Declaración previa obligatoria (`var x`; antes de `x = ...`).
- No redefinir en el mismo ámbito.
- Aritméticos (`+`, `-`, `*`, `/`, `%`): requieren numéricos; si se mezcla `INT`/`FLOAT`, el resultado es `FLOAT`.
- Comparación (`<`, `<=`, `>`, `>=`): sobre numéricos (promoción a `FLOAT` si aplica).
- Igualdad (`==` y `!=`): válida para numéricos, booleanos y strings.
- Lógicos (`and`, `or`, `not`, `/`, `&&`, `||` y `!`): sobre booleanos, con cortocircuito en `and`/`or`.
- Funciones/builtins: validar cantidad y tipo de argumentos; errores claros.

Integración con ANTLR (patrón Visitor)

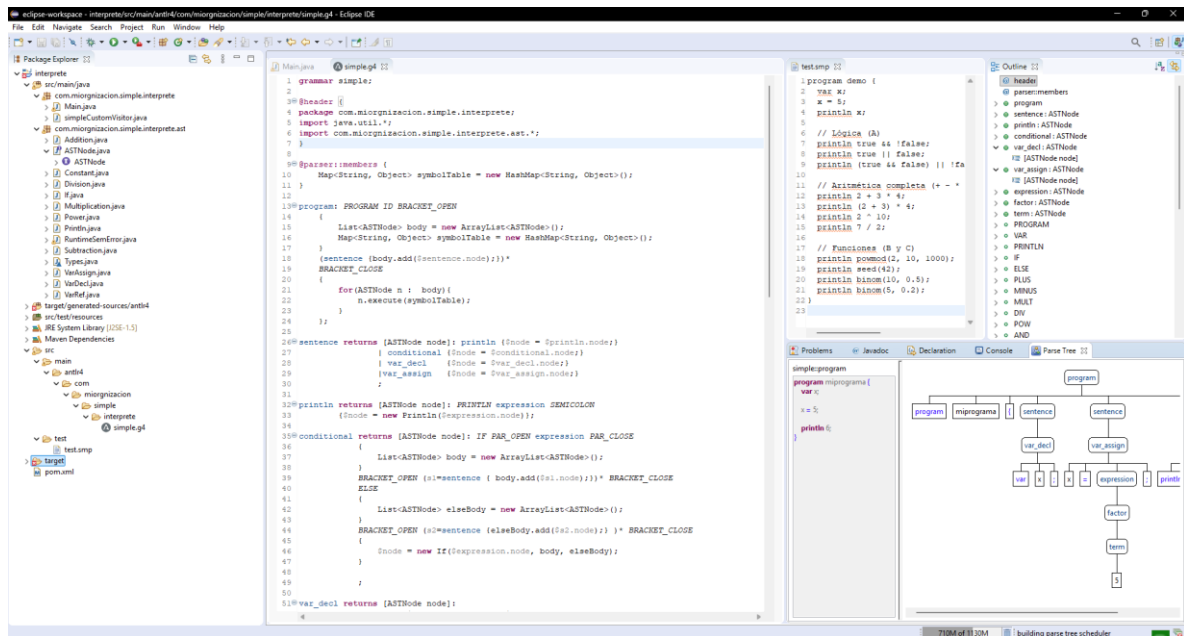
1. Bloques: en `visitBlock` hacer `pushScope()` al entrar y `popScope()` al salir.
2. Declaración (`var x`;): `define("x", símbolo)`; error si ya existe en el mismo ámbito.
3. Asignación (`x = expr`;): `resolve("x")`, evaluar `expr`, chequear tipo y asignar.
4. Expresiones: cada `visit...` devuelve un valor con tipo; aplicar promociones y validar operadores.
5. Salida (`println expr`;): evaluar y mostrar el resultado.

Mensajería de errores (formato sugerido)

- Sintaxis: Error de sintaxis en <línea>:<col> — <detalle>.
- Semántica:
 - <línea>:<col> Variable no declarada: x.
 - <línea>:<col> Tipos incompatibles en '+': STRING y INT.
 - <línea>:<col> powmod: módulo debe ser > 0.

Semántica:

- var x; → define("x") en el ámbito actual.
- x = 5; → resolve("x"), asignar INT(5).
- println x + 1; → evalúa x (INT) y 1 (INT) → INT(6) y lo imprime.



Prueba de funcionamiento final.